

idealFlow.cpp and data structures

Jay

1 Overview

This document explains the structure and logic of the C++ code `idealFlow.cpp`, which solves 1D ideal hydrodynamics using PETSc. The code reads simulation parameters from a JSON file, sets up the computational grid, initializes the solution, and advances it in time using a finite-volume method.

Note to self: Change diagrams to better ones!

- Charges: E, M_x
- Thermo variables: p, β, c_s^2
- primitive variables: e, u_x

2 Input Handling

The code reads simulation parameters from a JSON file (default: `inputs.json`). The relevant parameters include:

- `NX`: Number of grid points
- `xmin, xmax`: Domain boundaries
- `dof`: degrees of freedom per grid point/node, 7 in this case
- `CFL`:
- `iofilename`: Output HDF5 file name
- `initial_time, final_time`: Simulation time range
- `steps_per_print`: Output frequency

3 Main Data Structures

3.1 Class: EOS

The EOS class encapsulates the equation of state. It provides methods to compute pressure, speed of sound, temperature, and other thermodynamic quantities as functions of energy density e (ignore the baryon density ρ_B for now).

3.2 Struct: VischydroNode

This structure represents the hydrodynamic state at a single grid point. Its members (degrees of freedom at each node/grid point) are:

- E: Energy
- M: Momentum
- e: Energy density
- ux: Fluid velocity
- p: Pressure
- beta: Inverse temperature
- cs2: Speed of sound squared

It also provides methods for computing charges, fluxes, and derived quantities.

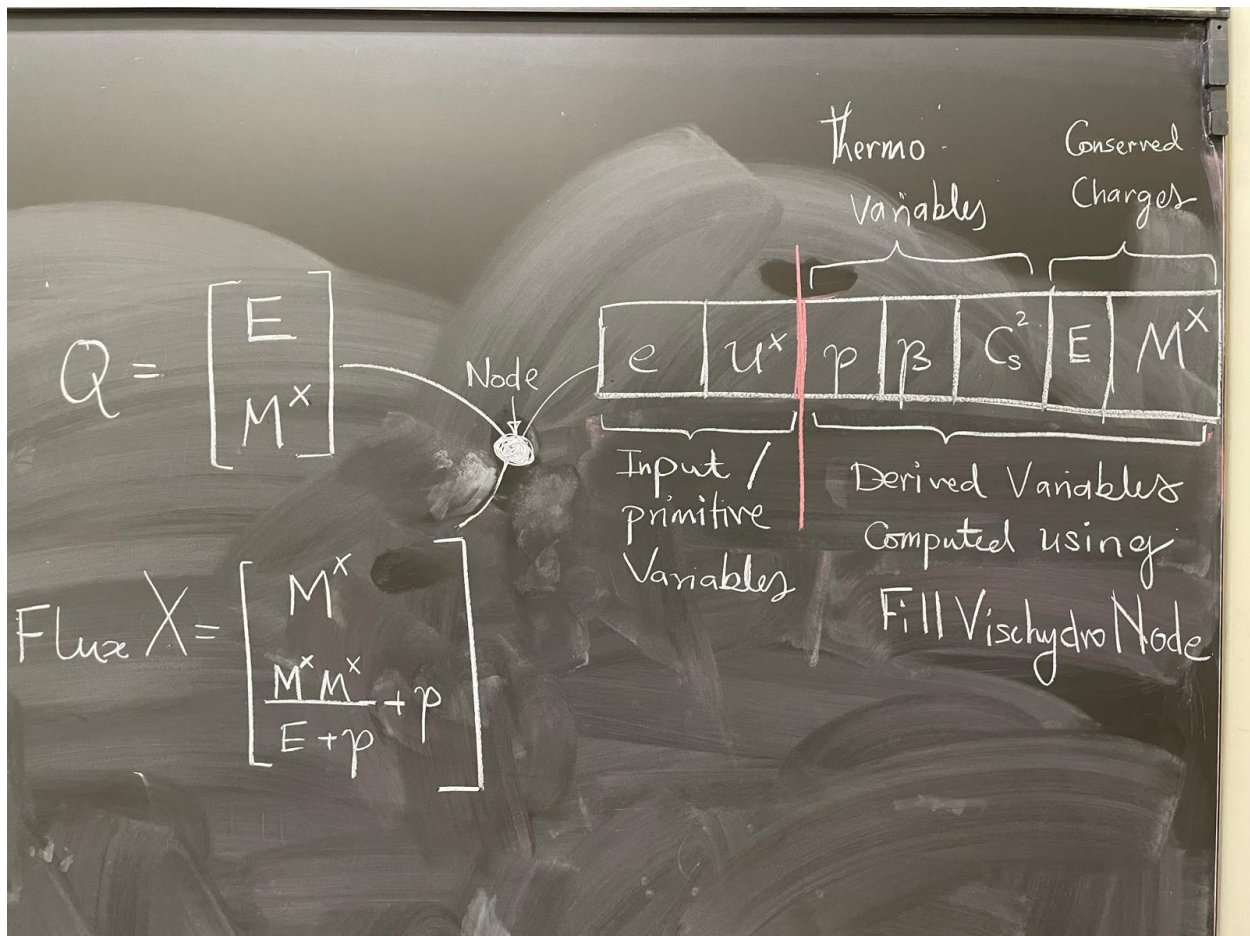


Figure 1: schematic diagram of a node and all the data on it

3.3 Class: Vischydro

This class manages the entire simulation, including:

- Reading inputs
- Setting up the PETSc DMDA grid
- Initializing solution vectors
- Managing the PETSc time stepper (TS)
- Outputting results to HDF5

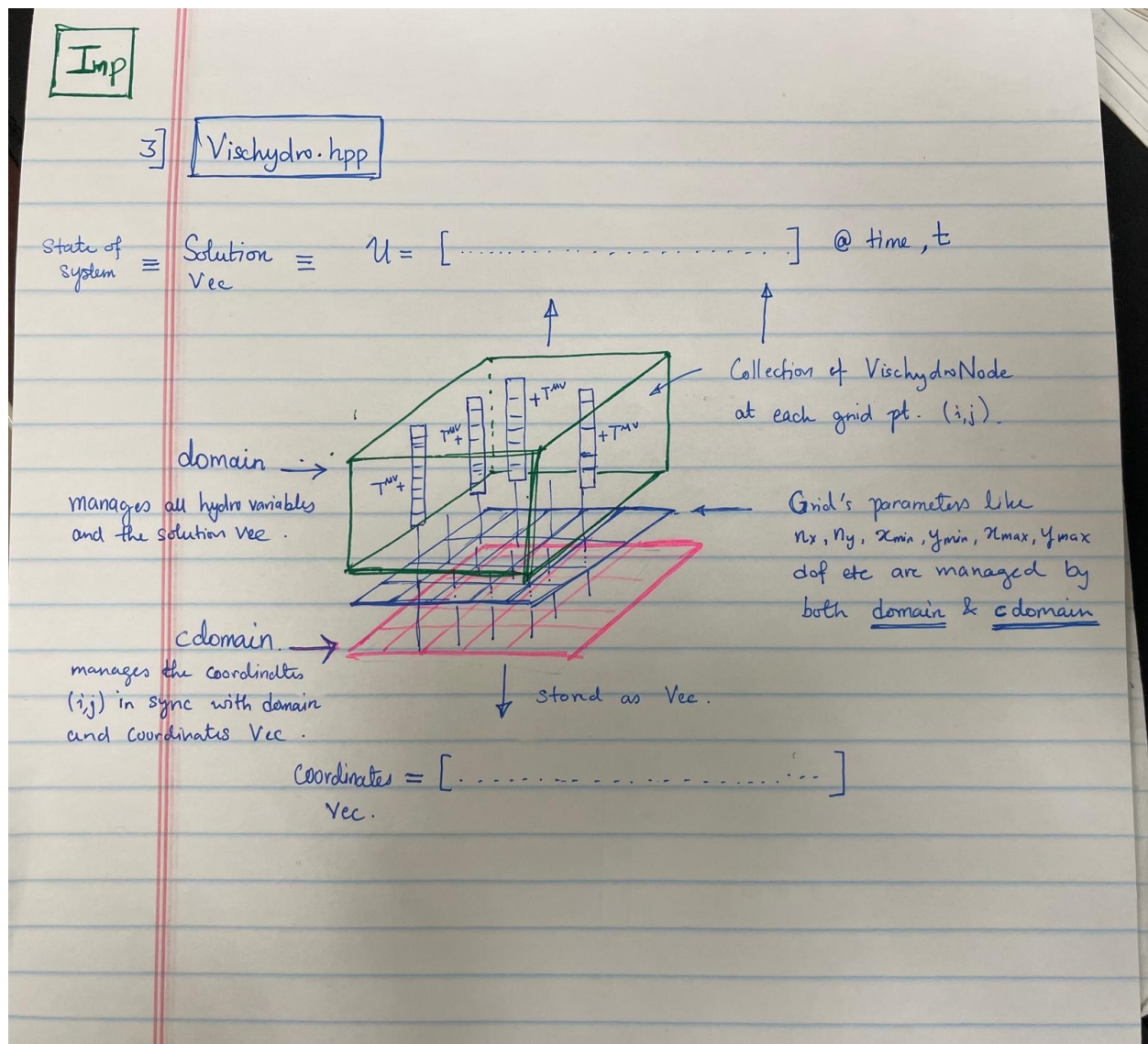


Figure 2: schematic diagram of how vischydro class

4 Algorithm

1. **Initialization:** Read inputs, set up grid, initialize solution with a Gaussian energy profile.
2. **Time Stepping:** Advance the solution using PETSc TS. At each step:
 - Compute fluxes using a slope limiter and Riemann solver.
 - Update solution vectors.
 - Output results at specified intervals.
3. **Post-processing:** Write final solution to HDF5.

5 Helper Functions

- **FillVischydroNode:** Starting with energy density ' e ' and EOS, this function fills the `VischydroNode` with charges E, M and thermo variables p, β, c_s^2
- **idealHydroCellSolve:** Given E, M , using Newton's method, this routine finds the energy density ' e ' and updates thermo variables
- **EulerRHSFunction:** Computes the right-hand side for time stepping.
- **VischydroMonitor:** Outputs solution at each time step.

6 Data Flow

- The main solution vector is a PETSc Vec of `VischydroNode` structs.
- At each time step, the solution is updated using fluxes computed from neighboring nodes.
- Output is written to an HDF5 file for post-processing and visualization.